```
"""
Runs experiments E1-E6 from the PBL report on Fashion-MNIST (PyTorch).
Usage (Colab, GPU runtime): !python run_experiments.py
Outputs go to ./results/: CSV tables (paste into the report) and PNG figures.
"""
import os, time, json, random
import numpy as np, torch, torch.nn as nn
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from torchvision import datasets, transforms
from sklearn.metrics import (roc_auc_score, precision_recall_fscore_support,
                             roc_curve, confusion_matrix, silhouette_score)
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import pandas as pd

SEED, ANOM, BS, EPOCHS, PATIENCE, LR, SIGMA = 0, 5, 128, 30, 5, 1e-3, 0.3
CLASSES = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat", "Sandal",
           "Shirt", "Sneaker", "Bag", "Ankle boot"]
dev = "cuda" if torch.cuda.is_available() else "cpu"
OUT = "results"; os.makedirs(OUT, exist_ok=True)

def seed_all(s=SEED):
    random.seed(s); np.random.seed(s); torch.manual_seed(s); torch.cuda.manual_se
seed_all()

# ---------------- data ----------------
tf = transforms.ToTensor()  # scales to [0,1]
tr = datasets.FashionMNIST("data", train=True, download=True, transform=tf)
te = datasets.FashionMNIST("data", train=False, download=True, transform=tf)
Xtr, ytr = tr.data.float().div(255).unsqueeze(1), tr.targets
Xte, yte = te.data.float().div(255).unsqueeze(1), te.targets

m = ytr != ANOM
Xn, yn = Xtr[m], ytr[m]
perm = torch.randperm(len(Xn), generator=torch.Generator().manual_seed(SEED))
nval = len(Xn) // 10
Xval, Xtrain = Xn[perm[:nval]], Xn[perm[nval:]]
Xtrain, Xval = Xtrain.to(dev), Xval.to(dev)
Xte, yte_d = Xte.to(dev), yte.to(dev)
is_anom = (yte == ANOM).numpy().astype(int)
g = torch.Generator(device="cpu").manual_seed(SEED)
noise_te = (torch.randn(Xte.shape, generator=g) * SIGMA).to(dev)
Xte_noisy = (Xte + noise_te).clamp(0, 1)
```

```
pd.DataFrame({"class": CLASSES,
              "train_count": np.bincount(tr.targets.numpy(), minlength=10),
              "test_count": np.bincount(te.targets.numpy(), minlength=10)}
             ).to_csv(f"{OUT}/class_counts.csv", index=False)
fig, ax = plt.subplots(10, 8, figsize=(8, 10))
for c in range(10):
    idx = (tr.targets == c).nonzero()[:8, 0]
    for j, i in enumerate(idx):
        ax[c, j].imshow(tr.data[i], cmap="gray"); ax[c, j].axis("off")
    ax[c, 0].set_title(CLASSES[c], fontsize=7, loc="left")
plt.tight_layout(); plt.savefig(f"{OUT}/fig_samples.png", dpi=120); plt.close()

# ---------------- models ----------------
class FCAE(nn.Module):
    def __init__(s, z=16):
        super().__init__()
        s.enc = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(),
                              nn.Linear(256, 64), nn.ReLU(), nn.Linear(64, z))
        s.dec = nn.Sequential(nn.Linear(z, 64), nn.ReLU(), nn.Linear(64, 256), nn
                              nn.Linear(256, 784), nn.Sigmoid())
    def encode(s, x): return s.enc(x)
    def forward(s, x): return s.dec(s.enc(x)).view(-1, 1, 28, 28)

class ConvAE(nn.Module):
    def __init__(s, z=16, bn=False, p=0.0):
        super().__init__()
        B = (lambda c: nn.BatchNorm2d(c)) if bn else (lambda c: nn.Identity())
        s.enc = nn.Sequential(nn.Conv2d(1, 16, 3, 2, 1), B(16), nn.ReLU(),
                              nn.Conv2d(16, 32, 3, 2, 1), B(32), nn.ReLU(),
                              nn.Flatten(), nn.Dropout(p), nn.Linear(32 * 49, z))
        s.fc = nn.Linear(z, 32 * 49)
        s.dec = nn.Sequential(nn.ConvTranspose2d(32, 16, 3, 2, 1, output_padding=
                              nn.ConvTranspose2d(16, 1, 3, 2, 1, output_padding=1
    def encode(s, x): return s.enc(x)
    def forward(s, x): return s.dec(s.fc(s.enc(x)).view(-1, 32, 7, 7))

nparams = lambda mod: sum(p.numel() for p in mod.parameters())

@torch.no_grad()
def recon(model, X, bs=1000):
    model.eval(); return torch.cat([model(X[i:i + bs]) for i in range(0, len(X),

@torch.no_grad()
def per_sample_mse(model, X, target=None):
    target = X if target is None else target
    r = recon(model, X)
```

```
    return ((r - target) ** 2).flatten(1).mean(1)

def train(model, wd=0.0, denoise=False):
    seed_all()
    model.to(dev)
    opt = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=wd)
    lossf = nn.MSELoss(); best, bad, best_state = 1e9, 0, None
    hist = {"train": [], "val": []}; t0 = time.time()
    for ep in range(EPOCHS):
        model.train(); idx = torch.randperm(len(Xtrain), device=dev); tl = 0
        for i in range(0, len(idx), BS):
            x = Xtrain[idx[i:i + BS]]
            xin = (x + SIGMA * torch.randn_like(x)).clamp(0, 1) if denoise else x
            loss = lossf(model(xin), x)
            opt.zero_grad(); loss.backward(); opt.step(); tl += loss.item() * len
        model.eval()
        with torch.no_grad():
            vin = (Xval + SIGMA * torch.randn_like(Xval)).clamp(0, 1) if denoise
            vl = lossf(recon(model, vin), Xval).item()
        hist["train"].append(tl / len(Xtrain)); hist["val"].append(vl)
        if vl < best - 1e-6:
            best, bad = vl, 0; best_state = {k: v.clone() for k, v in model.state
        else:
            bad += 1
            if bad >= PATIENCE: break
    model.load_state_dict(best_state)
    return hist, time.time() - t0

def evaluate(model, name, hist=None, ttime=None):
    err_te = per_sample_mse(model, Xte).cpu().numpy()
    err_val = per_sample_mse(model, Xval).cpu().numpy()
    err_tr = per_sample_mse(model, Xtrain).mean().item()
    tau = np.percentile(err_val, 95)
    pred = (err_te > tau).astype(int)
    p, r, f, _ = precision_recall_fscore_support(is_anom, pred, average="binary",
    return dict(model=name, params=nparams(model), train_time_s=ttime,
                val_mse=err_val.mean(), train_mse=err_tr,
                test_mse_normal=err_te[is_anom == 0].mean(),
                gap=err_val.mean() - err_tr, threshold=tau,
                precision=p, recall=r, f1=f, auc=roc_auc_score(is_anom, err_te)),

results, models, hists, errs = [], {}, {}, {}
def run(name, model, wd=0.0, denoise=False):
    print("training", name, flush=True)
    h, t = train(model, wd, denoise)
    row, e = evaluate(model, name, h, t)
    results.append(row); models[name] = model; hists[name] = h; errs[name] = e
```

```
    print({k: (round(v, 4) if isinstance(v, float) else v) for k, v in row.items(

# E1 + E5: A, B, C, D (latent 16)
run("A: FC-AE", FCAE(16))
run("B: Conv-AE", ConvAE(16))
run("C: Conv-AE + BN + Dropout + WD", ConvAE(16, bn=True, p=0.2), wd=1e-5)
run("D: DAE", ConvAE(16, bn=True, p=0.2), wd=1e-5, denoise=True)
# E2: ablation
run("Abl: +BatchNorm", ConvAE(16, bn=True))
run("Abl: +Dropout", ConvAE(16, p=0.2))
run("Abl: +WeightDecay", ConvAE(16), wd=1e-5)
# E3: bottleneck size (Model B)
for z in [2, 8, 32, 64]:
    run(f"Bottleneck z={z}", ConvAE(z))
pd.DataFrame(results).to_csv(f"{OUT}/all_results.csv", index=False)
res = {r["model"]: r for r in results}
bn_rows = [res["Bottleneck z=2"], res["Bottleneck z=8"], res["B: Conv-AE"],
           res["Bottleneck z=32"], res["Bottleneck z=64"]]
pd.DataFrame({"latent_dim": [2, 8, 16, 32, 64],
              "test_mse": [r["test_mse_normal"] for r in bn_rows],
              "auc": [r["auc"] for r in bn_rows]}).to_csv(f"{OUT}/E3_bottleneck.c

# E4: denoising
def psnr(mse): return 10 * np.log10(1 / mse)
rows = []
mse_noisy = ((Xte_noisy - Xte) ** 2).mean().item()
rows.append(("Noisy input (no model)", mse_noisy, psnr(mse_noisy)))
C, D = models["C: Conv-AE + BN + Dropout + WD"], models["D: DAE"]
for nm, mod in [("Model C", C), ("Model D (DAE)", D)]:
    mse = ((recon(mod, Xte_noisy) - Xte) ** 2).mean().item(); rows.append((nm, ms
pd.DataFrame(rows, columns=["model", "mse_vs_clean", "psnr_db"]).to_csv(f"{OUT}/E
n = 8; idx = torch.arange(n)
cd, nz, dn = Xte[idx], Xte_noisy[idx], recon(D, Xte_noisy[idx])
fig, ax = plt.subplots(3, n, figsize=(n * 1.2, 3.8))
for j in range(n):
    for i, (im, t) in enumerate([(cd, "clean"), (nz, "noisy"), (dn, "denoised")]):
        ax[i, j].imshow(im[j, 0].cpu(), cmap="gray"); ax[i, j].axis("off")
        if j == 0: ax[i, j].set_title(t, fontsize=8, loc="left")
plt.tight_layout(); plt.savefig(f"{OUT}/fig_denoising.png", dpi=120); plt.close()

# Figures: loss curves, gallery
fig, ax = plt.subplots(1, 4, figsize=(16, 3.2))
for a, nm in zip(ax, ["A: FC-AE", "B: Conv-AE", "C: Conv-AE + BN + Dropout + WD",
    a.plot(hists[nm]["train"], label="train"); a.plot(hists[nm]["val"], label="va
    a.set_title(nm, fontsize=8); a.set_xlabel("epoch"); a.legend()
plt.tight_layout(); plt.savefig(f"{OUT}/fig_loss_curves.png", dpi=120); plt.close
```

```
sel = torch.arange(0, 10)  # first 10 test images
fig, ax = plt.subplots(5, 10, figsize=(10, 5.4))
rows_im = [("original", Xte[sel])] + [(k, recon(models[k], Xte[sel])) for k in
           ["A: FC-AE", "B: Conv-AE", "C: Conv-AE + BN + Dropout + WD", "D: DAE"]
for i, (t, im) in enumerate(rows_im):
    for j in range(10):
        ax[i, j].imshow(im[j, 0].cpu(), cmap="gray"); ax[i, j].axis("off")
    ax[i, 0].set_title(t, fontsize=7, loc="left")
plt.tight_layout(); plt.savefig(f"{OUT}/fig_reconstructions.png", dpi=120); plt.c

# E5: best model by AUC among A-D
best_name = max(["A: FC-AE", "B: Conv-AE", "C: Conv-AE + BN + Dropout + WD", "D:
                 key=lambda k: res[k]["auc"])
e, r = errs[best_name], res[best_name]
print("best model by AUC:", best_name)
plt.figure(figsize=(5, 3.5))
plt.hist(e[is_anom == 0], bins=80, alpha=.6, label="normal", density=True)
plt.hist(e[is_anom == 1], bins=80, alpha=.6, label="Sandal (anomaly)", density=Tr
plt.axvline(r["threshold"], color="k", ls="--", label="threshold"); plt.legend()
plt.xlabel("reconstruction MSE"); plt.title(best_name, fontsize=8)
plt.tight_layout(); plt.savefig(f"{OUT}/fig_error_hist.png", dpi=120); plt.close(
fpr, tpr, _ = roc_curve(is_anom, e)
plt.figure(figsize=(4, 4)); plt.plot(fpr, tpr, label=f"AUC={r['auc']:.3f}"); plt.
plt.xlabel("FPR"); plt.ylabel("TPR"); plt.legend(); plt.tight_layout()
plt.savefig(f"{OUT}/fig_roc.png", dpi=120); plt.close()
cm = confusion_matrix(is_anom, (e > r["threshold"]).astype(int))
plt.figure(figsize=(3.5, 3.5)); plt.imshow(cm, cmap="Blues")
for i in range(2):
    for j in range(2): plt.text(j, i, cm[i, j], ha="center", va="center")
plt.xticks([0, 1], ["normal", "anomaly"]); plt.yticks([0, 1], ["normal", "anomaly
plt.xlabel("predicted"); plt.ylabel("true"); plt.tight_layout()
plt.savefig(f"{OUT}/fig_confusion.png", dpi=120); plt.close()

# Failure cases
mod = models[best_name]; rec = recon(mod, Xte).cpu()
nrm = np.where(is_anom == 0)[0]; an = np.where(is_anom == 1)[0]
fp = nrm[np.argsort(-e[nrm])[:5]]; fn = an[np.argsort(e[an])[:5]]
fig, ax = plt.subplots(4, 5, figsize=(6, 5))
for j in range(5):
    for i, (ids, src, t) in enumerate([(fp, Xte.cpu(), "FP input"), (fp, rec, "FP
                                       (fn, Xte.cpu(), "FN input"), (fn, rec, "FN
        ax[i, j].imshow(src[ids[j], 0], cmap="gray"); ax[i, j].axis("off")
        if j == 0: ax[i, j].set_title(t, fontsize=7, loc="left")
fp_labels = [CLASSES[int(yte[i])] for i in fp]
plt.tight_layout(); plt.savefig(f"{OUT}/fig_failure_cases.png", dpi=120); plt.clo

# E6: latent space of best model
```

```
with torch.no_grad():
    mod.eval(); Z = torch.cat([mod.encode(Xte[i:i + 1000]) for i in range(0, len(
y = yte.numpy()
sub = np.random.RandomState(SEED).choice(len(Z), 4000, replace=False)
P = PCA(2, random_state=SEED).fit_transform(Z[sub])
T = TSNE(2, random_state=SEED, init="pca").fit_transform(Z[sub])
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
for a, E, t in zip(ax, [P, T], ["PCA", "t-SNE"]):
    sc = a.scatter(E[:, 0], E[:, 1], c=y[sub], cmap="tab10", s=4); a.set_title(t)
ax[1].legend(handles=sc.legend_elements()[0], labels=CLASSES, fontsize=6, loc="be
plt.tight_layout(); plt.savefig(f"{OUT}/fig_latent.png", dpi=120); plt.close()
sil = silhouette_score(Z[sub], y[sub])
sil_normal = silhouette_score(Z[sub][y[sub] != ANOM], y[sub][y[sub] != ANOM])
cent = {c: Z[y == c].mean(0) for c in range(10)}
sand_dist = {CLASSES[c]: float(np.linalg.norm(cent[ANOM] - cent[c])) for c in ran

summary = dict(best_model=best_name, silhouette_all_classes=float(sil),
               silhouette_normal_only=float(sil_normal),
               sandal_centroid_distance=sand_dist,
               false_positive_classes=fp_labels, device=dev, seed=SEED)
json.dump(summary, open(f"{OUT}/summary.json", "w"), indent=2)
print(json.dumps(summary, indent=2))
print("Done. See ./results/")
```